// CHARACTER COUNT ---> 1ST

```
#include <stdio.h>
#include<string.h>
int main() {
  char d[20][20], f[20][25];
  int n, i, j, count;
  printf("Enter the number of frames: ");
  scanf("%d", &n);
  getchar();
  for(i=0; i<n; i++){
    printf("Enter frame %d: ", i+1);
    fgets(d[i], sizeof(d[i]), stdin);
    d[i][strcspn(d[i], "\n")] = '\0';
  }
  printf("\n***AT THE SENDER***\n");
  printf("Frames with character count: \n");
  for(i=0; i<n; i++){
    count = strlen(d[i])+1;
    f[i][0] = count+'0';
    f[i][1] = '\0';
    strcat(f[i], d[i]);
    printf("Frame %d: %s\n", i+1, f[i]);
  }
  printf("\nData Transmitted: ");
  for(i=0; i<n; i++)
    printf("%s", f[i]);
  printf("\n\n***AT THE RECEIVER***\n");
  printf("\nFrame after removing character count: \n");
  for(i=0; i<n; i++){
    count = f[i][0]-'0';
    for(j=0; j<count; j++)
      d[i][j] = f[i][j+1];
    d[i][count] = '\0';
    printf("Frame %d: %s\n", i+1, d[i]);
  }
  return 0;
}
```

Output:

```
Enter the number of frames: 3
Enter frame 1: 21
Enter frame 2: 4563
Enter frame 3: 237
***AT THE SENDER***
Frames with character count:
Frame 1: 321
Frame 2: 54563
Frame 3: 4237
Data Transmitted: 321545634237
***AT THE RECEIVER***
Frame after removing character count:
Frame 1: 21
Frame 2: 4563
Frame 3: 237
```

// BIT STUFFING --> 1st

```
#include <stdio.h>
#include<string.h>
int main() {
   char d[100];
   int i, count = 0;
   printf("Enter the bit stream: ");
   scanf("%s", d);
   printf("\nOriginal Data: %s\n", d);
   printf("\nTransmitted Frame: \n");
   printf("\n01111110");
   for(i=0; d[i]!='\0'; i++){
      printf("%c", d[i]);
      if(d[i] == '1'){
         count++;
         if(count == 5){
            printf("0");
            count = 0;
         }
      }
      else{
         count = 0;
      }
   }
   printf("01111110");
   return 0;
   return 0;
}
```

Output:

Enter the bit stream:
01111111101101110001101

Original Data:
01111111101101110001101

Transmitted Frame:

011111100111110111011011100011010
1111110

// CHARACTER STUFFING --> 1st

```
#include <stdio.h>
#include<string.h>
int main() {
   char d[100];
   int i, len;
   printf("Enter the data: ");
   scanf("%s", d);
   printf("\nOriginal data: %s\n", d);
   printf("\nCharacter Stuffed Frame:
\n");
   len = strlen(d);
   printf("FLAG");
   for(i=0; i<len; i++){
      if(strncmp(&d[i],"FLAG",4) == 0){
         printf("ESCFLAG");
         i += 3;
      }
      else if(strncmp(&d[i],"ESC",3) == 0){
         printf("ESCESC");
         i += 2;
      }
      else{
         printf("%c", d[i]);
      }
   }
   printf("FLAG");
   return 0;
}
```

Output:

```
Enter the data: RAFLAGSTESCSA

Original data: RAFLAGSTESCSA

Character Stuffed Frame:

FLAGRAESCFLAGSTESCESCSAFLAG
```

// CRC ---> 2nd

```
#include <stdio.h>
#include<string.h>
void crc(char d[], char divisor[]){
  int d_len, div_len, i, j;
  char temp[200], rem[200];
  d_len = strlen(d);
  div_len = strlen(divisor);
  strcpy(temp, d);
  for(i=0; i<div_len-1; i++)
    temp[d_len+i] = '0';
  temp[d_len+div_len-1] = '\0';
  strcpy(rem, temp);
  for(i=0; i<=strlen(temp)-div_len; i++){
    if(rem[i] == '1'){
      for(j=0; j<div_len; j++){
        if(rem[i+j] == divisor[j])
          rem[i+j] = '0';
        else
          rem[i+j] = '1';
      }
    }
  }
  printf("\nCRC Remainder: ");
  for(i=strlen(temp)-div_len+1;
i<strlen(temp); i++)
    printf("%c", rem[i]);
  printf("\nTransmitted codeword: %s",
d);
  for(i=strlen(temp)-div_len+1;
i<strlen(temp); i++)
    printf("%c", rem[i]);
  printf("\n");
}
int main() {
  char d[100], divisor[50];
  char ch;
  int choice;
  do{
    printf("\n***CRC MENU***\n");
    printf("1. CRC-12\n");
    printf("2. CRC-16\n");
    printf("3. CRC-CCITT\n");
    printf("4. Normal CRC(User-
defined)\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    if(choice == 5){
      printf("\nProgram terminated\n");
      break;
    }
    if(choice >= 1 && choice <= 4){
      printf("Enter binary data: ");
      scanf("%s", d);
    }
```

```
    switch(choice){
        case 1: printf("\nUsing CRC-12 Generator Polynomial\n");
        crc(d, "1100000001111");
        break;

        case 2: printf("\nUsing CRC-16 Generator Polynomail\n");
        crc(d, "11000000000000101");
        break;

        case 3: printf("\nUsing CRC-CCITT Generator Polynomial\n");
        crc(d, "10001000000100001");
        break;

        case 4: printf("Enter Generator Polynomial(binary): ");
        scanf("%s", divisor);
        crc(d, divisor);
        break;

        default: printf("\nInvalid Choice\n");
    }
    printf("\nDo you want to continue? [Y/N]: ");
    scanf(" %c", &ch);
  }while(ch == 'Y' || ch == 'y');
  printf("\nThank you!!");
  return 0;
}
```

Output:

```
***CRC MENU***
1. CRC-12
2. CRC-16
3. CRC-CCITT
4. Normal CRC(User-defined)
5. Exit
Enter your choice: 1
Enter binary data: 1001101

Using CRC-12 Generator Polynomial

CRC Remainder: 001011110010
Transmitted codeword: 1001101001011110010

Do you want to continue? [Y/N]: y

***CRC MENU***
1. CRC-12
2. CRC-16
3. CRC-CCITT
4. Normal CRC(User-defined)
5. Exit
Enter your choice: 2
Enter binary data: 1001101
```

Using CRC-16 Generator Polynomail

CRC Remainder: 0000000110101110

Transmitted codeword: 1001101000000110101110

Do you want to continue? [Y/N]: y

CRC MENU

1. CRC-12

2. CRC-16

3. CRC-CCITT

4. Normal CRC(User-defined)

5. Exit

Enter your choice: 3

Enter binary data: 1011001

Using CRC-CCITT Generator Polynomial

CRC Remainder: 1100101111011100

Transmitted codeword: 1011001110010111011100

Do you want to continue? [Y/N]: y

CRC MENU

1. CRC-12

2. CRC-16

3. CRC-CCITT

4. Normal CRC(User-defined)

5. Exit

Enter your choice: 4

Enter binary data: 1011001

Enter Generator Polynomial(binary): 1001

CRC Remainder: 011

Transmitted codeword: 1011001011

Do you want to continue? [Y/N]: y

CRC MENU

1. CRC-12

2. CRC-16

3. CRC-CCITT

4. Normal CRC(User-defined)

5. Exit

Enter your choice: 5

Program terminated

Thank you!!

// SLIDING WINDOW --> 3rd

```
#include <stdio.h>

int main() {
  int n, ws;
  int ack, i, j;
  int base = 1, nf;
  printf("Enter number of frames: ");
  scanf("%d", &n);
  printf("Enter window size: ");
  scanf("%d", &ws);
  nf = ws+1;
  while(base <= n){
    printf("\nSender window: ");
    for(i=base; i<base+ws && i<=n; i++)
      printf("%d ", i);
    printf("\n");
    for(i=base; i<base+ws && i<=n; i++){
      printf("\nIs ACK received for frame %d? (1=yes, 0=no): ", i);
      scanf("%d", &ack);
      if(ack == 1){
        printf("ACK recieved for frame %d\n", i);
        if(nf<=n){
          printf("Sending frame %d\n", nf);
          nf++;
        }
        base++;
      }
      else{
        printf("\nFrame %d lost\n", i);
        printf("Go-Back-N Retransmission:\n");
        for(j=i; j<nf; j++)
          printf("Retransmitting frame %d\n", j);
        base = i;
        break;
      }
    }
  }
  printf("\nAll frames transmitted successfully");
  return 0;
}
```

// DIJKSTRAS ---> 4th

```
#include <stdio.h>
#define MAX 10
#define INF 9999
int main() {
  int graph[MAX][MAX], dist[MAX], visited[MAX];
  int n, source;
  int i, j, u, min;
  printf("Enter the number of vertices: ");
  scanf("%d", &n);
  printf("Enter the adjacency matrix: \n");
  for(i=0; i<n; i++){
    for(j=0; j<n; j++){
      scanf("%d", &graph[i][j]);
      if(i != j && graph[i][j] == 0)
        graph[i][j] = INF;
    }
  }
  printf("Enter the source vertex(0 to %d): ", n-1);
  scanf("%d", &source);
  for(i=0; i<n; i++){
    dist[i] = graph[source][i];
    visited[i] = 0;
  }
  dist[source] = 0;
  visited[source] = 1;
  for(i=1; i<n; i++){
    min = INF;
    for(j=0; j<n; j++){
      if(!visited[j] && dist[j] < min){
        min = dist[j];
        u = j;
      }
    }
    visited[u] = 1;
    for(j=0; j<n; j++){
      if(!visited[j] && (dist[u]+graph[u][j] < dist[j])){
        dist[j] = dist[u]+graph[u][j];
      }
    }
  }
  printf("\nShortest distances from vertex %d:\n", source);
  for(i=0; i<n; i++)
    printf("vertex %d --> vertex %d = %d\n", source, i, dist[i]);
  return 0;
}
```

Output:

```
Enter the number of vertices: 5
Enter the adjacency matrix:
0 10 0 30 100
```

10 0 50 0 0

0 50 0 20 10

30 0 20 0 60

100 0 10 60 0

Enter the source vertex(0 to 4): 3

Shortest distances from vertex 3:

vertex 3 --> vertex 0 = 30

vertex 3 --> vertex 1 = 40

vertex 3 --> vertex 2 = 20

vertex 3 --> vertex 3 = 0

vertex 3 --> vertex 4 = 30

// BROADCAST ---> 5th

```
#include <stdio.h>
#define MAX 100
int main() {
  int graph[MAX][MAX], visited[MAX];
  int queue[MAX];
  int n, source;
  int i, j;
  int front = 0, rear = 0;
  printf("Enter number of hosts: ");;
  scanf("%d", &n);
  printf("Enter adjacency matrix:\n");
  for(i=0; i<n; i++){
    for(j=0; j<n; j++){
      scanf("%d", &graph[i][j]);
    }
  }
  printf("Enter the source host(0 to %d): ", n-1);
  scanf("%d", &source);
  for(i=0; i<n; i++)
    visited[i] = 0;
  visited[source] = 1;
  queue[rear++] = source;
  printf("\nBroadcast Tree Edges:\n");
  while(front < rear){
    int current = queue[front++];
    for(i=0; i<n; i++){
      if(graph[current][i] == 1 && !visited[i]){
        visited[i] = 1;
        queue[rear++] = i;
        printf("%d --> %d\n", current, i);
      }
    }
  }
  printf("\nBroadcast completed successfully");
  return 0;
}
```

Output:

```
Enter number of hosts: 5
Enter adjacency matrix:
0 1 0 0 1
0 0 1 0 0
1 0 1 1 0
0 1 0 1 1
1 0 0 0 1
Enter the source host(0 to 4): 0

Broadcast Tree Edges:
0 --> 1
0 --> 4
1 --> 2
2 --> 3
Broadcast completed successfully
```

// DVR ---> 6th

```
#include <stdio.h>
#define MAX 10
#define INF 999
int main() {
  int cost[MAX][MAX], dist[MAX][MAX], next[MAX][MAX];
  int n, i, j, k, changed;
  printf("Enter no. of nodes: ");
  scanf("%d", &n);
  printf("Enter cost matrix (9999 for nolink):\n");
  for(i=0; i<n; i++){
    for(j=0; j<n; j++){
      scanf("%d", &cost[i][j]);
      dist[i][j] = cost[i][j];
      if(i == j)
        next[i][j] = -1;
      else if(cost[i][j] != INF)
        next[i][j] = j;
      else
        next[i][j] = -1;
    }
  }
  do{
    changed = 0;
    for(i=0; i<n; i++){
      for(j=0; j<n; j++){
        for(k=0; k<n; k++){
          if(dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k]+dist[k][j]){
            dist[i][j] = dist[i][k]+dist[k][j];
            next[i][j] = next[i][k];
            changed = 1;
          }
        }
      }
    }
  }while(changed);
  for(i=0; i<n; i++){
    printf("\nRouting table for node: %d\n", i+1);
    printf("Destination \t NextHop \t Cost\n");
    for(j=0; j<n; j++){
      if(i == j)
        printf("\t%d \t\t\t Self \t\t 0\n", j+1);
      else if(dist[i][j] == INF)
        printf("\t%d \t\t\t INF \n", j+1);
      else
        printf("\t%d \t\t\t %d \t\t\t %d\n", j+1, next[i][j], dist[i][j]);
    }
  }
  return 0;
}
```

Output:

Enter no. of nodes: 4

Enter cost matrix (9999 for nolink):

0 2 6 9999

2 0 1 5

6 1 0 1

9999 5 1 0

Routing table for node: 1

Destination	NextHop	Cost
1	Self	0
2	1	2
3	1	3
4	1	4

Routing table for node: 2

Destination	NextHop	Cost
1	0	2
2	Self	0
3	2	1
4	2	2

Routing table for node: 3

Destination	NextHop	Cost
1	1	3
2	1	1
3	Self	0
4	3	1

Routing table for node: 4

Destination	NextHop	Cost
1	2	4
2	2	2
3	2	1
4	Self	0

// ENCRYPTION & DECRYPTION ----> 7th

```
#include<stdio.h>
#include<string.h>

int main() {
  char d[100], enc[100], dec[100];
  int enc_key, dec_key;
  int i;
  printf("---Source System---\n");
  printf("Enter the data: ");
  scanf("%s", d);
  printf("Enter Encryption key: ");
  scanf("%d", &enc_key);
  for(i=0; d[i]!='\0'; i++){
    if(d[i] >= 'A' && d[i] <= 'Z')
      enc[i] = ((d[i]-'A'+enc_key)%26) +
'A';
    else if(d[i] >= 'a' && d[i] <= 'z')
      enc[i] = ((d[i]-'a'+enc_key)%26) +
'a';
    else
      enc[i] = d[i];
  }
  enc[i] = '\0';
  printf("Encrypted data: %s\n", enc);
  printf("\n---Destination System---\n");
  printf("Enter Decrypt key: ");
  scanf("%d", &dec_key);
  for(i=0; enc[i]!='\0'; i++){
    if(enc[i] >= 'A' && enc[i] <= 'Z')
      dec[i] = ((enc[i]-'A'-
(dec_key%26)+26)%26) + 'A';
    else if(enc[i] >= 'a' && enc[i] <= 'z')
      dec[i] = ((enc[i]-'a'-
(dec_key%26)+26)%26) + 'a';
    else
      dec[i] = enc[i];
  }
  dec[i] = '\0';
  printf("Decrypted data: %s", dec);
  return 0;
}
```

Output:

```
---Source System---
Enter the data: Computer
Enter Encryption key: 3
Encrypted data: Frpsxwhu

---Destination System---
Enter Decrypt key: 3
Decrypted data: Computer
```

// LEAKY BUCKET ---> 8th

```
#include <stdio.h>

int main() {
   int buc_size, output_rate;
   int incoming, bucket = 0;
   int n, i;
   int transmitted, dropped;
   printf("Enter bucket size: ");
   scanf("%d", &buc_size);
   printf("Enter output rate: ");
   scanf("%d", &output_rate);
   printf("Enter no. of time interval: ");
   scanf("%d", &n);
   printf("\n---Leaky Bucket Simulation---\n");
   printf("Time \t Incoming \t Transmitted \t Dropped \t Remaining\n");
   printf("-------------------------------------------------------\n");
   for(i=1; i<=n; i++){
      printf("\nEnter incoming packets at time %d: ", i);
      scanf("%d", &incoming);
      dropped = 0;
      transmitted = 0;
      if(bucket+incoming > buc_size){
         dropped = (bucket+incoming)-buc_size;
         bucket = buc_size;
      }
      else
         bucket += incoming;
      if(bucket >= output_rate){
         transmitted = output_rate;
         bucket -= output_rate;
      }
      else{
         transmitted = bucket;
         bucket = 0;
      }
      printf("%d \t %d \t\t %d \t\t %d \t\t %d\n", i, incoming, transmitted, dropped, bucket);
   }
   while(bucket > 0){
      if(bucket >= output_rate){
         transmitted = output_rate;
         bucket -= output_rate;
      }
      else{
         transmitted = bucket;
         bucket = 0;
      }
      printf("\n %d \t 0 \t\t %d \t\t 0 \t\t %d\n", i, transmitted, bucket);
```

```
    i++;
  }
  return 0;
}
```

Output:

```
Enter bucket size: 10
Enter output rate: 3
Enter no. of time interval: 5

---Leaky Bucket Simulation---
Time    Incoming        Transmitted
Dropped         Remaining
-------------------------------------------------
----

Enter incoming packets at time 1: 4
1         4         3         0         1

Enter incoming packets at time 2: 5
2         5         3         0         3

Enter incoming packets at time 3: 8
3         8         3         1         7

Enter incoming packets at time 4: 3
4         3         3         0         7

Enter incoming packets at time 5: 6
5         6         3         3         7
```

6	0	3	0	4
7	0	3	0	1
8	0	1	0	0

// FRAME SORTING ---> 9th

```
#include <stdio.h>
struct Frame{
  int seq;
  char d[100];
};
int main() {
  struct Frame f[20], temp;
  int n, i, j;
  printf("Enter the no. of frames: ");
  scanf("%d", &n);
  for(i=0; i<n; i++){
    printf("Frame %d sequence: ", i+1);
    scanf("%d", &f[i].seq);
    printf("Frame %d data: ", i+1);
    scanf("%s", f[i].d);
  }
  for(i=0; i<n-1; i++){
    for(j=i+1; j<n; j++){
      if(f[i].seq > f[j].seq){
        temp = f[i];
        f[i] = f[j];
        f[j] = temp;
      }
    }
  }
  printf("\nFrames after sorting:\n");
  printf("Sequence no. \t Data\n");
  for(i=0; i<n; i++){
    printf("%d \t\t %s\n", f[i].seq, f[i].d);
  }
  return 0;
}
```

Output:

```
Enter the no. of frames: 5
Frame 1 sequence: 4
Frame 1 data: E
Frame 2 sequence: 2
Frame 2 data: B
Frame 3 sequence: 1
Frame 3 data:
A
Frame 4 sequence: 5
Frame 4 data: G
Frame 5 sequence: 3
Frame 5 data: C

Frames after sorting:
Sequence no.        Data
1                   A
2                   B
3                   C
4                   E
5                   G
```